

Jeff Wood

Greater Seattle, WA | [linkedin.com/in/scripted-world](https://www.linkedin.com/in/scripted-world) | github.com/scriptedworld

ABOUT

A smile, or the spark of understanding, is what motivates me: either when someone comes to understand something I have helped teach, or when something I have built saves them time and effort. I have spent my career building automation, frameworks, services, and knowledge for my coworkers. I apply test-engineering discipline to developer tooling, from validation across thousands of hosts and artifact pipelines in regulated environments to self-service infrastructure and verification tooling for coding agents. I am at my best turning processes that depend on an expert into systems other engineers can run, understand, and rely on themselves.

TECHNICAL SKILLS

Languages: Python, Ruby, Java, JavaScript, C++, Rust, Go

Infrastructure: Ansible, Puppet, Docker, k3s/k3d, Kubernetes, RHEL8+/CentOS, Debian, AWS (S3, Kinesis, Redshift)

Securing developer pipelines: GitLab CI, Artifactory, JFrog Xray, SBOM/OCI analysis, CVE remediation, static analysis

Observability: Datadog, operational metrics and dashboards, live queue/state instrumentation

AI & Agent Tooling: Agent orchestration, MCP tool-server development, shell-AST policy gates, structured-output contracts, enforced TDD and per-file coverage gates

Data: SQL (MySQL, Postgres, Redshift), ETL pipeline design, streaming ingestion, Segment

Practice: Test planning and frameworks, dependency and release management, vulnerability remediation, code review and merge gating, regulated change control

EXPERIENCE

Staff Systems Engineer at ServiceNow · Aug 2021 – Present · Seattle, WA

Global Cloud Services: Test Engineering, then Pipeline Architecture.

Fleet validation framework · An Ansible-driven validation framework that runs server- and client-side checks across thousands of fleet hosts worldwide. It runs multiple times a day across the org, wired into production deployment tickets before and after scheduled maintenance of inter-datacenter routing, DNS, LDAP, and certificates. Across the full tenure:

- Maintained framework development and CI/CD and reviewed team and user-community contributions as a required approver for merges to main. Service owners also ran it against their own maintenance.
- As sole engineer on its Python 3.6 → 3.12 modernization, added RHEL9 and SELinux support while preserving the test-running interface. Replaced an expert-operated bootstrap with 2 lines, download and run the bootstrap, then run the tests, and cut the on-host preparation payload by about 80%; only the controller paid the Ansible-wheelhouse cost.
- Extended the framework from operator-mediated results to self-service reporting in central development tooling. Added export and upload and designed REST ingestion and dashboard, exposing one host through multi-datacenter environments. Results could be grouped by service, network segment, test suite, host OS, or other test metadata and compared across up to 3 runs for regression and trend analysis; held final sign-off.

Also at ServiceNow

- Provided direct and secondary support inside US IL4 and IL5 segmented environments: testing and debugging the fleet validation framework; maintaining and mitigating vulnerabilities in the company's self-hosted Artifactory, including version updates and host replacements and migrations.
- Took ownership of the org's Python toolchain and built a reconciliation pipeline that ran for years: it tracked upstream releases, regenerated its build image, and filled missing RPMs across Python 3.9 → 3.14 and 3 OS targets.
- Built supply-chain analysis tooling for CVEs due in each audit window. It pulled candidate containers and compared Anchore and Trivy SBOMs to identify either the software release that fixed a CVE or the first candidate carrying a sufficiently new version of the flagged package, avoiding brittle RPM filename and version-string parsing.
- Built and stood up sk8park: one REST call returned a kubeconfig for an ephemeral k3s cluster, reclaimed on release or TTL. External Secrets Operator came pre-wired to the company's internal secret-management system, so Helm charts and services could retrieve their secrets without running on lab or production fabrics.

continued...

- Built tempenv, ending recurring manual cleanup across hundreds of hosts by redirecting cache and temporary files into ephemeral environments removed through lifecycle hooks. Precompiled per-OS wheelhouses install offline with no compilation or resolution, keeping one approved artifact byte-identical across disconnected federal environments and verifiable by hash.

Senior Software Engineer at Moz · Dec 2019 – Aug 2021 · Seattle, WA

- Maintained and extended Linkscope, Moz's Python web-scale link-index crawler, and the qless Redis-backed job queue beneath it, while a data scientist tuned the ranking model. Root-caused an indexing failure that had resisted diagnosis, unblocking reclamation of millions of stale objects from a multi-terabyte RIAK cluster.

Software Development Engineer, Data Engineering at Moz · Apr 2016 – Dec 2019 · Seattle, WA

- Designed and built the config-driven ETL platform feeding Amazon Redshift from internal and third-party systems: 16 data streams with schema management, transforms, and S3 I/O. Another team later ported it from on-prem to AWS largely unchanged; it was still running on my last day at Moz.
- Removed the engineering queue from adding business metrics: built a template-driven pipeline where analysts defined metrics as SQL files and the system executed and snapshotted them on schedule as Redshift time series.
- Instrumented an existing SPA for feature usage and timing, then built the Kinesis pipeline carrying its client-posted events into Redshift.
- Taught Python to the BI analysts I worked with, keeping pipeline development and analysis in one language.

Senior Software Development Engineer in Test at Moz · Nov 2013 – Mar 2016 · Seattle, WA

- Built an in-house visual regression service when CSS layout regressions depended on someone noticing and off-the-shelf tools were not widely available. It diffed page snapshots against historic baselines with a tuned significance threshold, then rendered animated comparisons so reviewers could see layout jitter at a glance.
- Built Ruby test and administration tooling for the payments and subscriptions platform, including coverage of the card-updater ingestion path.
- Built a distributed system generating realistic web and social traffic to test the web crawler and search-score prediction pipeline. Its HTTP Markov-chain service supplied post and article content across distinct personas, creating the synthetic corpora the traffic carried.
- Replaced a Ruby web application, moving middle-tier processing into its JavaScript front end.
- Taught 6 QAEs and SDETs Ruby, testing methodology, and CLI tooling.

Senior Software Development Engineer in Test at Cequent · Oct 2010 – Nov 2013 · Seattle, WA

- Led handset testing for a major carrier's Caller ID rollout. With the carrier's engineers, built a 300-element test list covering in-network and roaming handoffs while preserving Caller ID delivery and in-call quality; ran the suite at 7 sites, each representing a different vendor's core-network hardware.
- Mentored SDETs and QAEs in Ruby, shell, and testing technique.
- Built and ran dist at Cequent, the load-testing framework later published as the public MIT-licensed dist / dist-graph project. It proved projected national load could run on any 1 of 4 host sets after a data-center and rack failure, with 20% headroom.

Software Development Engineer in Test II at Amazon.com · Dec 2004 – Oct 2010 · Seattle, WA

- Built test suites, internal tools, and event-stream pipelines across 5 groups including Log Ingestion and Client-Side Metrics; started on Perl, wrote TestNG suites against Java services, then introduced Ruby-based test frameworks: more readable and maintainable than the Perl they replaced, and I got more done in them.

Earlier: DONOBI, Squarerigger, Medicalibration · Web applications and Perl data ingestion at DONOBI; C++ middleware and VB utility libraries at Squarerigger; C++ hardware-interface and imaging tools at Medicalibration.

SELECTED ENGINEERING

Quality tools: Bolt / Wrench / Toolbox (*Rust, Go, Python, and YAML*) · Together, Bolt, Wrench, and Toolbox turn a suite of quality tools into one pass/fail gate. Bolt normalizes common tool outputs through configurable adapters and aggregates their decisions. Wrench provides schema-validated TOML, JSON, and YAML I/O in separate Rust, Go, and Python implementations, with bundled schemas and parity checks. Toolbox supplies the common and language-specific checks, tasks, and adapters Bolt runs.

Claude tools: Infobot / Qwark / Skid (*Go and Python*) · Infobot, Qwark, and Skid work at Claude's native control surfaces: its status line, PreToolUse hooks, and MCP. Infobot exposes project and session state, context use, rate-limit pace, subagent activity, and counterfactual API cost; its Go rewrite cut event latency from 40.4 ms to 11.8 ms. Qwark gates proposed shell commands against configurable patterns and records each decision with its matching rules and rule-set digest. Skid exposes queued MCP voice generation and identifies which agent needs attention. Claude lets you dictate; Skid lets it talk back.